



통합 게이밍 대시보드 및 하드웨어 매칭 플랫폼

SYNCRIG

AI 협업을 통한 소프트웨어 생애 주기 실습 발표

학번: 2021663046

이름: 이건우

<https://github.com/lgw323/webserviceplatform>

<https://syncrig.lgw323.com/>

(2주차)

게이머의 스트레스를 데이터로 해결하는 최적화 플랫폼



✓ 복잡한 그래픽 옵션의 늪

최신 게임들의 수십가지가 넘는 세부 설정을 직접 건드려야 하는 피로도

✓ 데이터 파편화와 비효율

여러 플랫폼에 플레이 기록 조회와 게임별 최적화 설정 탐색이 분리되어 발생하는 탐색 피로도

✓ 하드웨어 성능의 낭비

수동 설정 시 발생하는 성능 손실 수치 (예: 약 25%의 프레임 저하 발생) 시각화

(3주차)

12주 프로젝트 로드맵

1 WEEK 1-2

기획 완료

1 WEEK 3-4

명세서 수립

1 WEEK 5-6

플로우&아키텍처 설계

1 WEEK 7-8

개발 시작

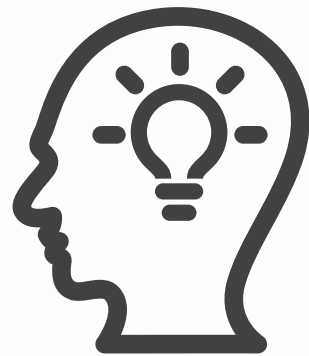
1 WEEK 9-10

통합 테스트

1 WEEK 11-12

배포 준비 및 배포

1



기획 및 아이디어

프로젝트 컨셉 확립 및 요구사항 분석

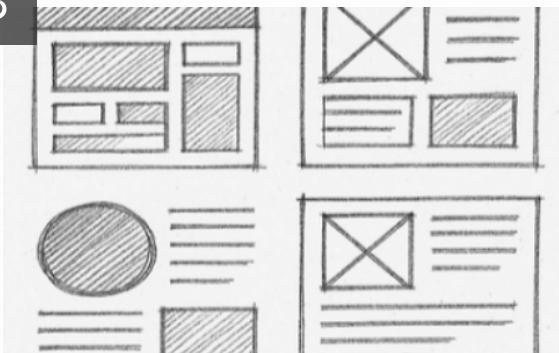
2



시스템 설계

아키텍처 설계 및 DB 구조 정의

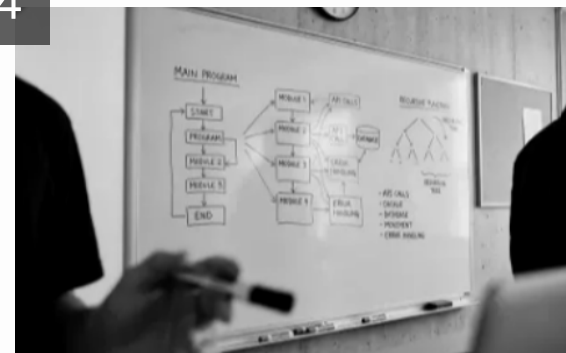
3



UI/UX 개발

사용자 인터페이스 개발

4



백엔드 개발

서버 및 API 개발

(4주차)

개발의 나침반이 되어주는 요구사항 명세서 작성



어떻게 작동할 것인가?



기능명세서



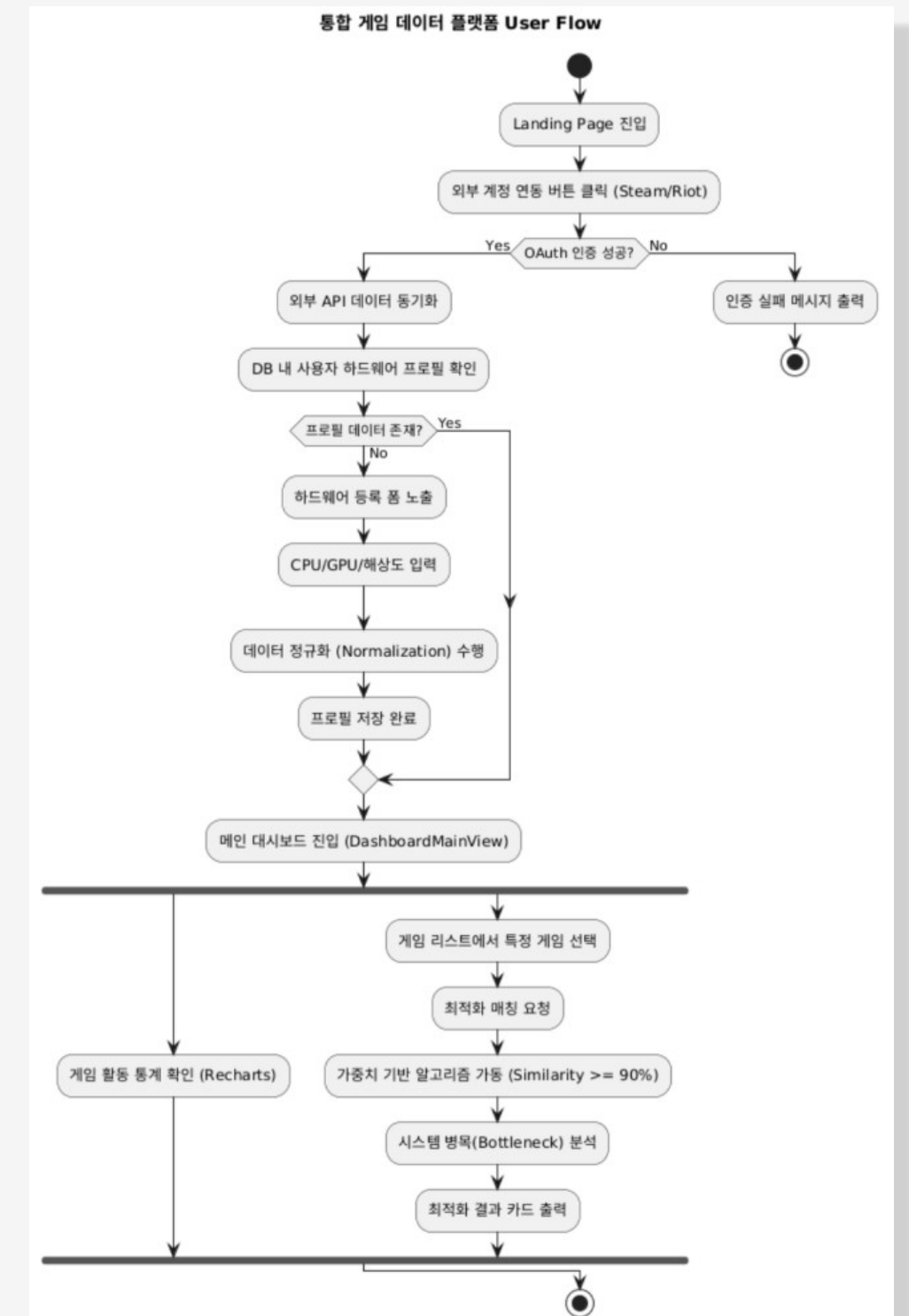
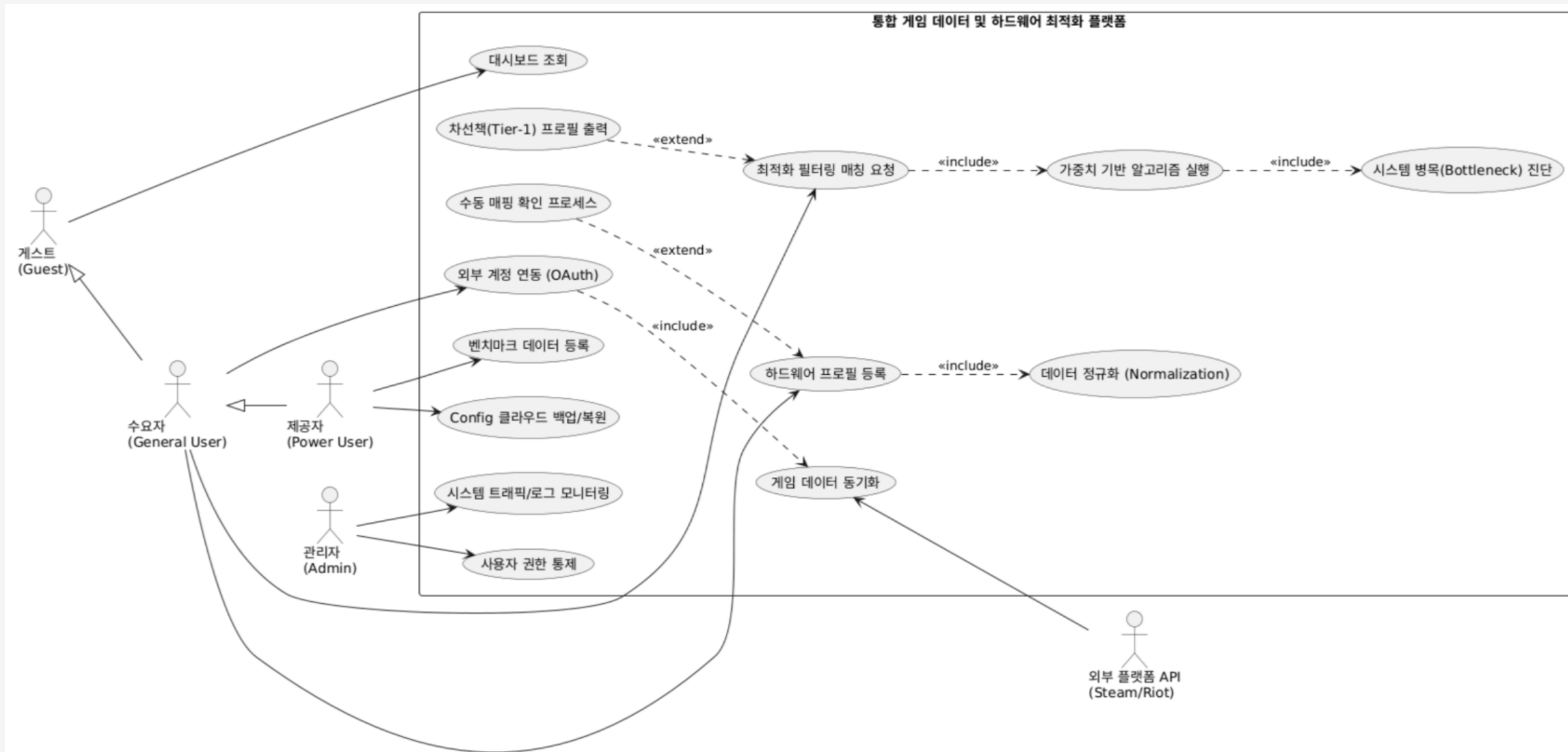
무엇을 만들 것인가?



요구사항 명세서

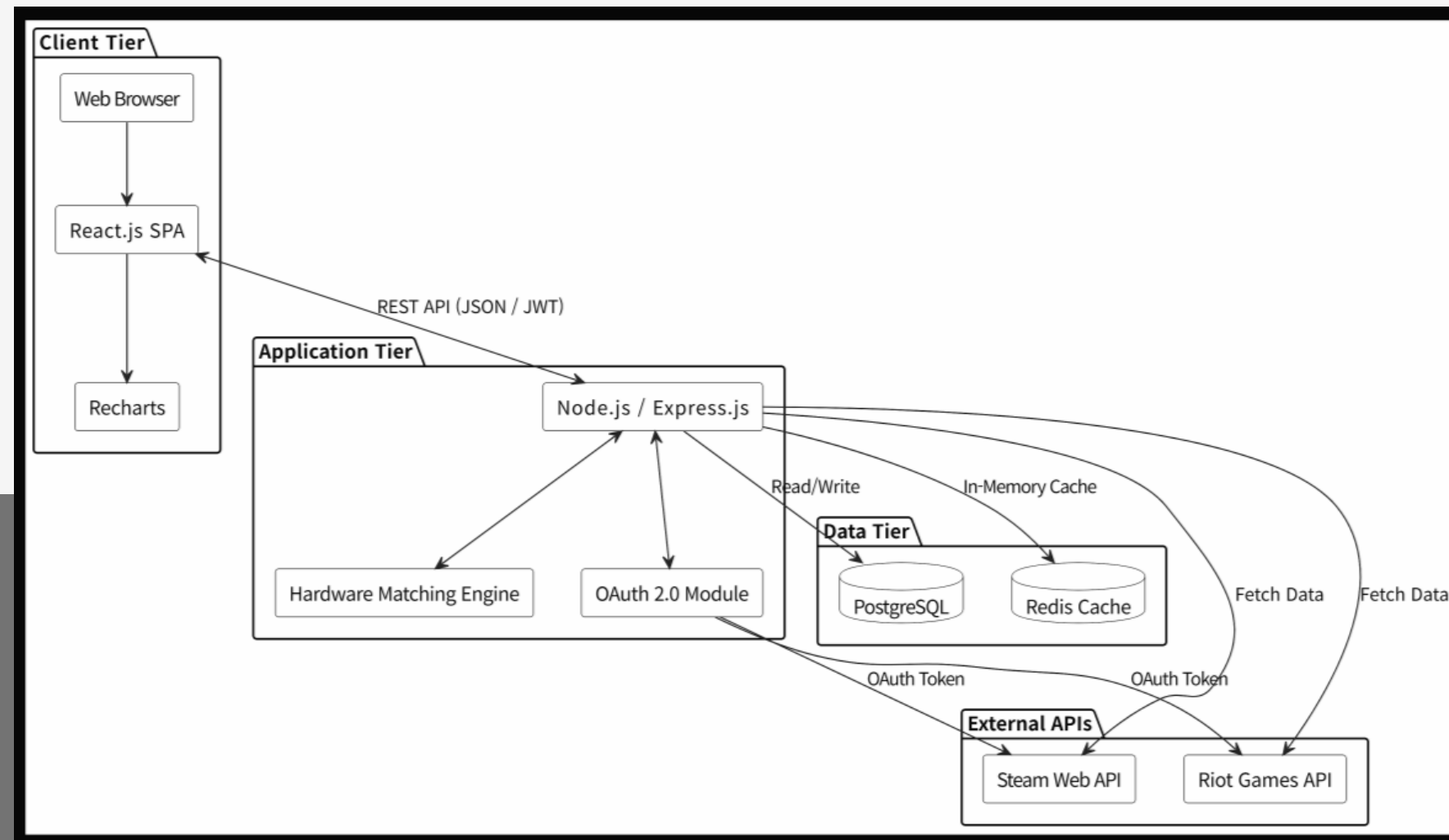
(5주차)

USE CASE 및 플로우 설계

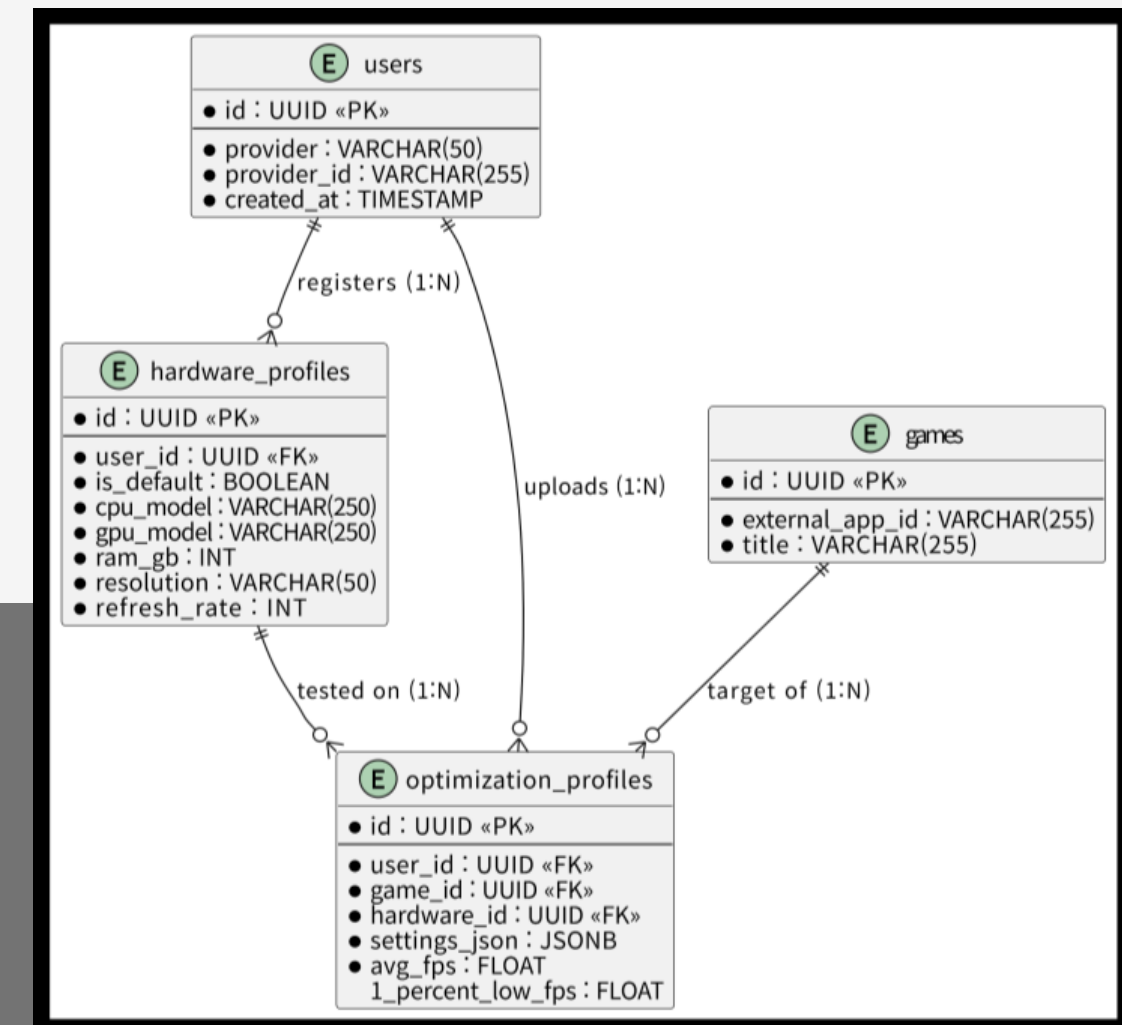


(6주차)

시스템 아키텍처 및 ERD



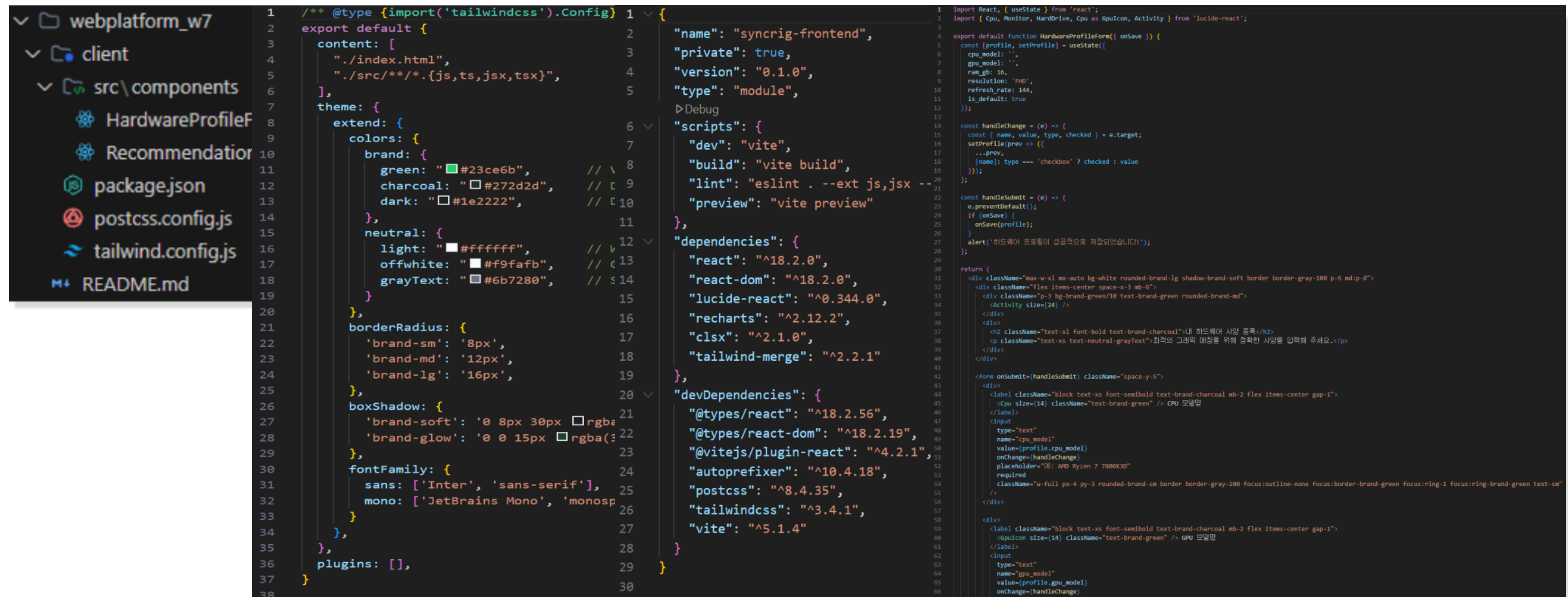
상세 아키텍처



ERD 구성

(7주차)

화면을 구성하는 프론트엔드 UI 컴포넌트 개발



(8주차)

데이터 관리를 위한 백엔드 서버 및 DB 구축

webplatform_w8

server

src

config

db.js

models

schema.sql

package.json

server.js

README.md

```
1 import pg from 'pg';
2 import dotenv from 'dotenv';
3
4 dotenv.config();
5
6 const { Pool } = pg;
7
8 // Establish database pool with environment variables
9 const pool = new Pool({
10   connectionString: process.env.DATABASE_URL || 'postgresql://postgres:postgres@localhost:5432/syncrig',
11   ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
12 });
13
14 pool.on('connect', () => {
15   console.log('[SYNCRIG] Connected to the PostgreSQL database.');
```

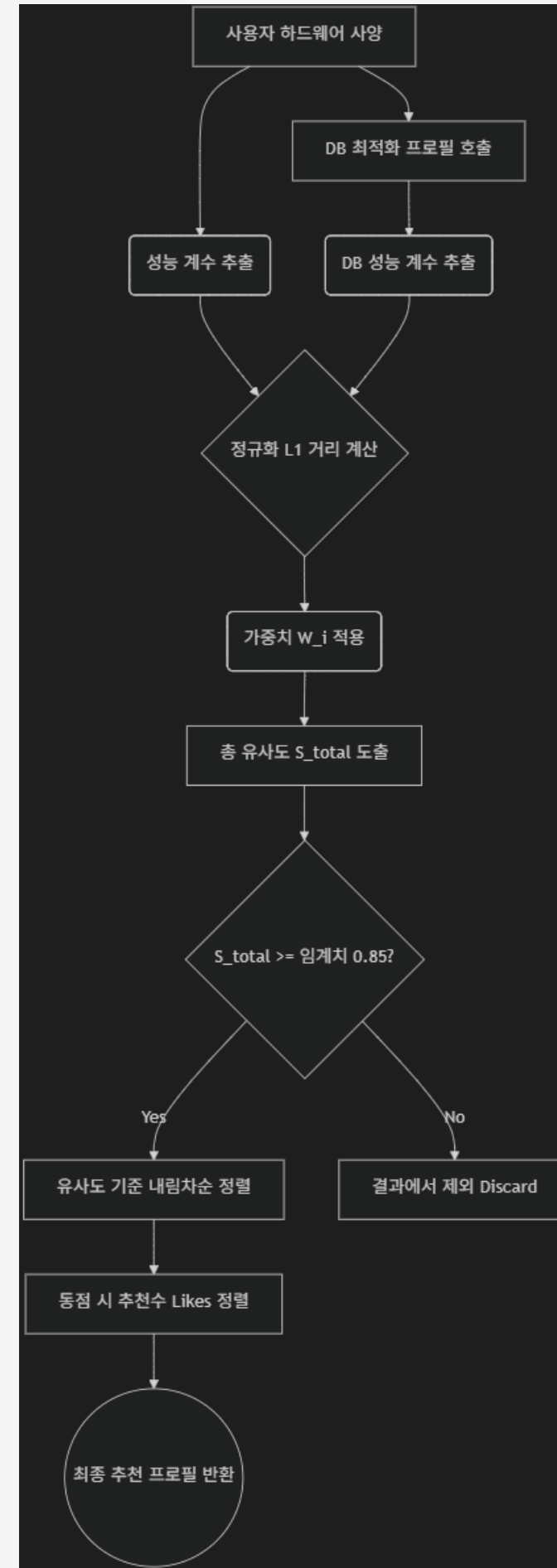
```
1 -- SYNCRIG Database Schema Definition
2 -- PostgreSQL Syntax
3
4 CREATE EXTENSION IF NOT EXISTS "uuid-oss";
5
6 -- 1. users table
7 CREATE TABLE IF NOT EXISTS users (
8   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
9   provider VARCHAR(50) NOT NULL,          -- e.g., 'steam', 'riot'
10  provider_id VARCHAR(255) NOT NULL,       -- platform specific user identification
11  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
12  UNIQUE(provider, provider_id)
13 );
14
15 -- 2. hardware_profiles table
16 CREATE TABLE IF NOT EXISTS hardware_profiles (
17   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
18   user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
19   is_default BOOLEAN DEFAULT false NOT NULL,
20   cpu_model VARCHAR(250) NOT NULL,        -- e.g., 'AMD Ryzen 7 7800X3D'
21   gpu_model VARCHAR(250) NOT NULL,        -- e.g., 'NVIDIA GeForce RTX 4070 SUPER'
22   ram_gb INT NOT NULL,                   -- e.g., 16, 32
23   resolution VARCHAR(50) NOT NULL,       -- e.g., 'FHD', 'QHD', '4K'
24   refresh_rate INT NOT NULL,             -- e.g., 144, 240
25   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
26 );
27
28 -- 3. games table
29 CREATE TABLE IF NOT EXISTS games (
30   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
31   external_app_id VARCHAR(255) NOT NULL UNIQUE, -- Steam AppID or Riot Match ID
32   title VARCHAR(255) NOT NULL,           -- e.g., 'Cyberpunk 2077'
33   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
34 );
35
36 -- 4. optimization_profiles table
37 CREATE TABLE IF NOT EXISTS optimization_profiles (
38   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
39   user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
40   game_id UUID NOT NULL REFERENCES games(id) ON DELETE CASCADE,
41   hardware_id UUID NOT NULL REFERENCES hardware_profiles(id) ON DELETE CASCADE,
42   settings_json JSONB NOT NULL,          -- detailed graphical settings
43   avg_fps FLOAT NOT NULL,                -- average fps measured
44   one_percent_low_fps FLOAT,             -- 1% low fps (optional)
45   game_version VARCHAR(50) NOT NULL,     -- e.g., 'v2.12'
46   likes INT DEFAULT 0 NOT NULL,
47   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
48 );
49
50 -- Index optimization for similarity matching
51 CREATE INDEX IF NOT EXISTS idx_hardware_gpu ON hardware_profiles(gpu_model);
52 CREATE INDEX IF NOT EXISTS idx_optimization_game ON optimization_profiles(game_id);
53
```

```
1 import express from 'express';
2 import cors from 'cors';
3 import dotenv from 'dotenv';
4 import pg from 'pg';
5
6 dotenv.config();
7
8 const app = express();
9 const PORT = process.env.PORT || 5000;
10
11 // Middleware
12 app.use(cors());
13 app.use(express.json());
14
15 // Database connection status route
16 app.get('/api/health', (req, res) => {
17   res.json({
18     status: 'up',
19     timestamp: new Date().toISOString(),
20     service: 'SYNCRIG API Server'
21   });
22 });
23
24 // User API Routes Placeholder
25 app.post('/api/v1/users/hardware-profiles', (req, res) => {
26   const { cpu_model, gpu_model, ram_gb, resolution, refresh_rate } = req.body;
27   if (!cpu_model || !gpu_model || !ram_gb || !resolution || !refresh_rate) {
28     return res.status(400).json({
29       status: 'error',
30       message: '필수 하드웨어 정보(CPU, GPU, RAM, 해상도, 주사율)가 누락되었습니다.'
31     });
32   }
33
34   // mock database save logic
35   res.status(201).json({
36     status: 'success',
37     message: '하드웨어 프로파일 성공적으로 저장되었습니다.',
38     data: {
39       profile_id: '550e8400-e29b-41d4-a716-446655440000',
40       hardware: { cpu_model, gpu_model, ram_gb, resolution, refresh_rate }
41     }
42   });
43 });
44
45 // Express Global Error Handler
46 app.use((err, req, res, next) => {
47   console.error(err.stack);
48   res.status(500).json({
49     status: 'error',
50     message: '서버 내부 에러가 발생했습니다. 잠시 후 다시 시도해 주세요.'
51   });
52 });
53
54 app.listen(PORT, () => {
55   console.log(`[SYNCRIG] Server is running on port ${PORT}`);
56 }
```

(9주차)

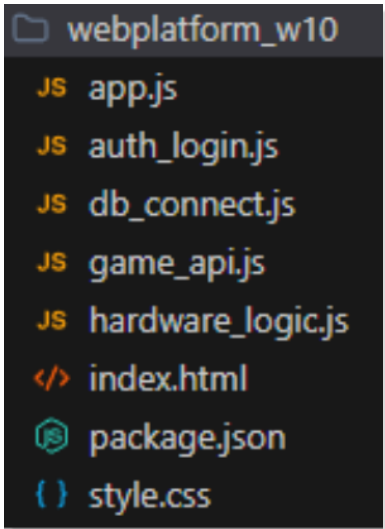
유사도 매칭을 위한 알고리즘 개발

```
1  /**
2   * SYNCRIG Matching Engine v1.0
3   * Calculates similarity scores between a user's hardware profile and uploaded profiles.
4   * Weighted: GPU (50%), CPU (30%), RAM (10%), Resolution (10%)
5   */
6
7   // GPU Generation and tier mapping for simplified matching
8   const GPU_TIERS = {
9     '4090': 100, '4080': 90, '4070': 80, '4060': 70,
10    '3090': 85, '3080': 75, '3070': 65, '3060': 55,
11    '2080': 50, '2070': 45, '2060': 35,
12  };
13
14   const CPU_TIERS = {
15     '7950x': 100, '7800x': 95, '7700x': 85, '7600x': 75,
16     '14900k': 100, '13900k': 95, '14700k': 90, '13700k': 85, '14600k': 80, '13600k': 75,
17     '5800x': 70, '5600x': 60,
18  };
19
20   function parseGpuScore(gpuName) {
21     const nameLower = gpuName.toLowerCase();
22     for (const [key, val] of Object.entries(GPU_TIERS)) {
23       if (nameLower.includes(key)) return val;
24     }
25     return 50; // default medium score
26   }
27
28   function parseCpuScore(cpuName) {
29     const nameLower = cpuName.toLowerCase();
30     for (const [key, val] of Object.entries(CPU_TIERS)) {
31       if (nameLower.includes(key)) return val;
32     }
33     return 50; // default medium score
34   }
35
36   /**
37    * Calculates a similarity score between a user hardware spec and a database record.
38    * Returns a value between 0.0 and 1.0.
39    */
40   export function calculateSimilarity(userSpec, dbSpec) {
41     let score = 0;
42
43     // 1. GPU (50% weight)
44     const userGpuScore = parseGpuScore(userSpec.gpu_model);
45     const dbGpuScore = parseGpuScore(dbSpec.gpu_model);
46     const gpuDiff = Math.abs(userGpuScore - dbGpuScore);
47     const gpuSimilarity = Math.max(0, 1 - gpuDiff / 100);
48     score += gpuSimilarity * 0.5;
49
50     // 2. CPU (30% weight)
51     const userCpuScore = parseCpuScore(userSpec.cpu_model);
52     const dbCpuScore = parseCpuScore(dbSpec.cpu_model);
53     const cpuDiff = Math.abs(userCpuScore - dbCpuScore);
54     const cpuSimilarity = Math.max(0, 1 - cpuDiff / 100);
55     score += cpuSimilarity * 0.3;
```



(중간 점검)

중간 AI 점검



[Architecture Refactoring] 프로젝트 디렉토리 구조 현업 표준화 및 계층 분리

현재 프로젝트의 파일들이 역할 구분 없이 하나의 루트 디렉토리(`old_messy_structure_for_ppt`)에 존재되어 있어, 향후 코드 베이스가 커질 경우 심각한 유지보수성 저하 및 스파게티 코드가 발생할 위험이 높습니다.

이에 따라, 프론트엔드와 백엔드를 물리적으로 분리하고 각각 현업에서 권장하는 표준 아키텍처(React 플더 구조 및 Express MVC 패턴)를 도입 하는 대규모 리팩토링을 제안합니다.

User Review Required

WARNING

본 리팩토링은 거의 모든 파일의 경로를 이동시키므로 기존 파일 간의 `import/require` 경로가 전부 깨지게 됩니다. 반드시 별도의 `refactor/architecture` 브랜치를 생성하여 작업해야 하며, 프론트와 백엔드를 동시에 띄우는 환경 스크립트 수정이 동반되어야 합니다. 진행하시겠습니까?

문제 상황 분석 (현재 구조)

- 프론트엔드 코드(`index.html`, `app.js`, `style.css`)와 백엔드 코드(`db_connect.js`, `auth_login.js`)가 섞여 있습니다.
- 역할(View, Logic, Data)에 따른 파일 분리가 되어 있지 않습니다.
- 프론트엔드 의존성(라이브러리)과 백엔드 의존성이 하나의 `package.json`에 섞여 의존성 충돌 위험이 있습니다.

Proposed Changes (제안하는 개선 아키텍처)

현업에서 가장 널리 쓰이는 **Client / Server** 완전 분리형 아키텍처를 제안합니다.

1. Root 레벨 분리

가장 먼저 전체 코드를 프론트엔드 영역과 백엔드 영역으로 분리합니다.

[NEW] `client/` : 프론트엔드 소스코드 (React / Vite 기반)

[NEW] `server/` : 백엔드 소스코드 (Node.js / Express 기반)

2. Client (Frontend) 세부 아키텍처 도입

`app.js`와 `index.html`을 모던 웹 개발 표준에 맞게 플더별로 쪼개어 재배치합니다.

[MODIFY] `client/src/components/` : 재사용 가능한 UI 컴포넌트 모음

[MODIFY] `client/src/pages/` : 라우팅되는 화면 단위 컨테이너

[MODIFY] `client/src/hooks/` : 커스텀 훅 (비즈니스 로직 분리)

[MODIFY] `client/src/api/` : 서버 API 통신 관련 로직 격리

3. Server (Backend) MVC 패턴 도입

`db_connect.js`와 `auth_login.js` 등을 MVC(Model-View-Controller) 패턴에 따라 라우팅과 로직으로 나눕니다.

[MODIFY] `server/src/routes/` : API 엔드포인트 URL 경로 정의

[MODIFY] `server/src/controllers/` : 요청(Request)을 받아 처리하는 핵심 비즈니스 로직

[MODIFY] `server/src/models/` : 데이터베이스(PostgreSQL) 쿼리 및 스키마

[MODIFY] `server/src/middlewares/` : 인증(JWT) 및 여러 처리 등 중간 개입 로직

[MODIFY] `server/src/config/` : DB 연결 등 환경 설정

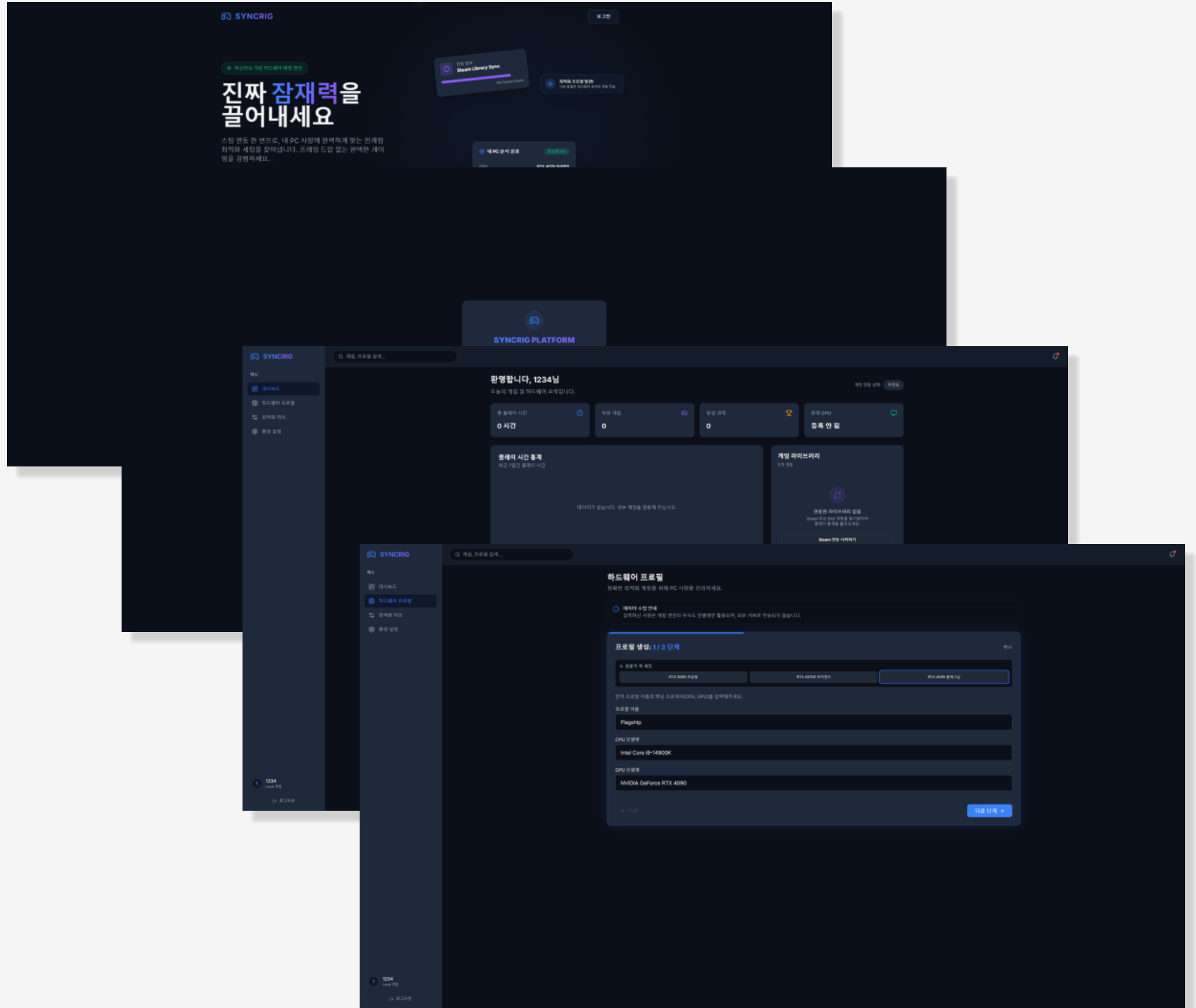
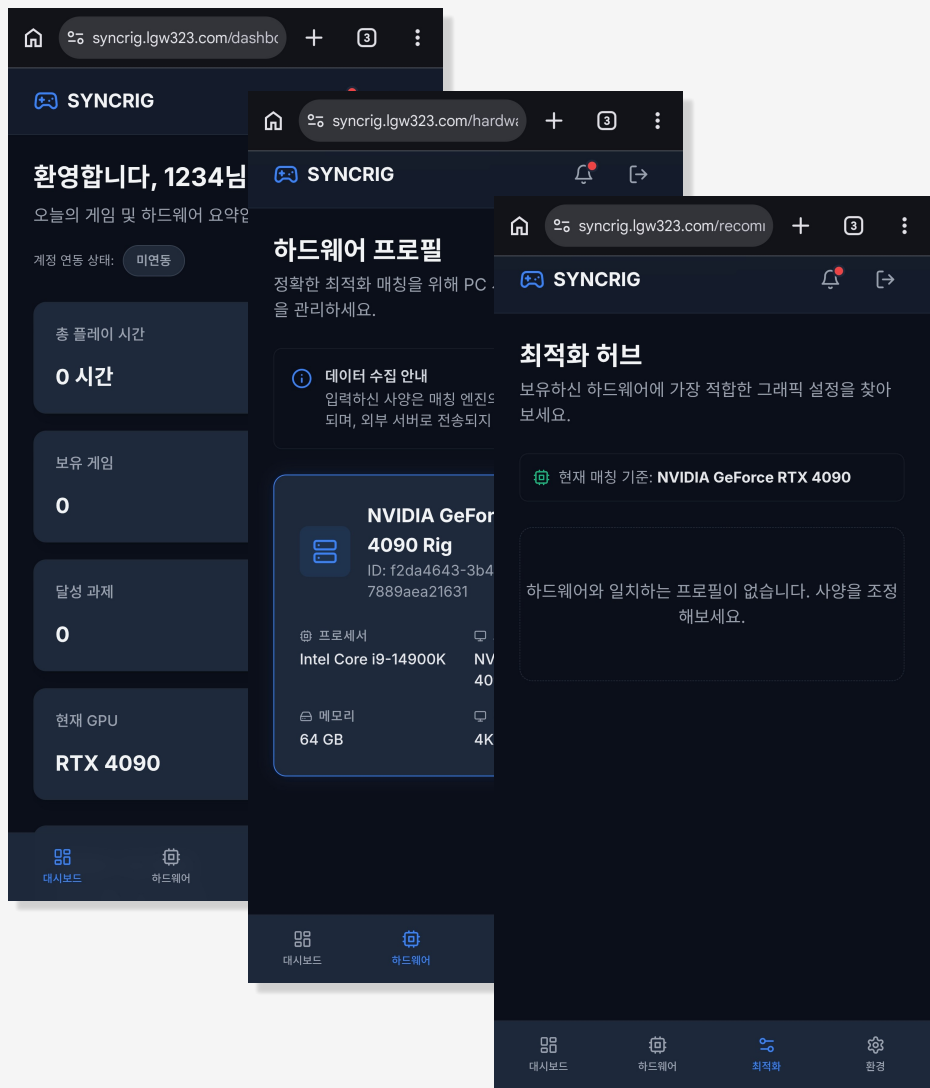
Verification Plan

1. 파일 마이그레이션: 제안된 구조에 맞게 기존 파일들을 전부 이동.

2. 모듈 시스템 재검바: 프론트는 ES Modules(`import`), 서버는 CommonJS(`require`)로 환경 분리.

(10주차)

프론트/백엔드 통합 및 대시보드 시각화



(11&12주차)

시스템 안정화 및 배포

Production Deployment

Repository

Instant Rollback

Visit

No Screenshot Available

Deployment

webplatform-l10u872xx-dlrjsdn0518-8450s-projects.vercel.app

Domains

webplatform-sable.vercel.app

Status

Created

Ready

31m ago by lgw323

Source

main

2dbfa02 upload#10: webplatform_w10/client/src/App.jsx - 대시보드 내 Steam/Riot 계정 연...

Deployment Settings

Recommendations

Build Multiple Deployments Simultaneously

Never wait for a queued build.

Get builds up to 40% faster

Switch to a bigger build machine.

Prevent Frontend-Backend Mismatches

Automatically sync client and server versions to avoid deployment conflicts.

Build Settings

On-Demand Concurrent Builds

Disabled

Build Machine

Standard 4 vCPU 8 GB Memory

Prioritize Production Builds

Enabled

Runtime Settings

Fluid Compute

Enabled

Deployment Protection

Standard Protection

Function Region

Function CPU

Standard 1 vCPU 2 GB Memory

Skew Protection

Disabled

Node.js Version

24.x

Cold Start Prevention

Disabled

webplatform									
Deployments									
All Branches		All Authors		All Environments		Select Date Range		Status 6/7	
docs: EXPLANATION.md에서 불필요한 실제 사용법 내용 제거	Ready 19s	Preview	68773cd	feat/외부계정연동				7m ago	
docs: README.md에 버전 관리 정책 및 커밋 컨벤션 가이드 추가	Ready 17s	Preview	96849e8	feat/외부계정연동				10h ago	
feat: 스팀 및 라이엇 외부 계정 연동(Passport OAuth/OpenID) 구현	Ready 19s	Preview	ec12e81	feat/외부계정연동				10h ago	
docs: Cloudflare 도메인 연결 Vercel 가이드 추가	Ready 16s	Production	6d953ef	main				10h ago	
docs: EXPLANATION.md에 Vercel DB 및 환경변수 완벽 세팅 가이드 추가	Ready 25s	Production		Redeploy of 4UpMUo5pM				10h ago	
docs: EXPLANATION.md에 Vercel DB 및 환경변수 완벽 세팅 가이드 추가	Ready 17s	Production	e6ea951	main				10h ago	
fix: 지명적인 인증 오류(JWT Secret 키 불일치) 및 재진 파일(authMiddleware, db.js) 복구, #1 Steam 실제 API 연동 기능 구현(steamService), 기타 데이터 항목(모바일 반응형, npm 스크립트...	Ready 15s	Production	6a48568	main				11h ago	
upload#13: webplatform_w13/EXPLANATION.md - 서비스 개선 사항을 비유와 함께 쉽게 설명하는 설명서 신규 작성 (카레 출력중, 화이트보드, 반찬 킬름, 가짜 돈보가, 가짜 초인종, 열리레...	Ready 17s	Production	73f69e8	main				23h ago	
Merge branch 'fix/critical-ux-bugs-and-service-stability' into main	Ready 18s	Production	d76a15f	main				2d ago	
Merge pull request #1 from feature/web-service-robustness	Ready 17s	Production	3c89a2b	main				Jun 1	
upload#13: README.md - 13주차 별첨근성(WCAG 2.0) 및 SEO 최적화 적용 결과 문서화 및 프로젝트 메인 로드업 13주차 업데이트	Ready 14s	Production	a586fa	main				Jun 1	
upload#12: README.md - 1주차부터 12주차까지의 최종 프로젝트 산출물 통합 결과 보고서 작성	Ready 14s	Production	1b8f625	main				May 22	
upload#10: webplatform_w10/client/src/App.jsx - 대시보드 내 Steam/Riot 계정 연동 시 기존 세션을 덮어쓰지 않고 계정 연동 상태만 추가하도록 로직 개선	Ready 14s	Production	2dbfa82	main				May 22	
upload#10: webplatform_w10/client/package.json - Vercel 빌드 환경 esbuild 리눅스 바이너리 누락 오류 해결을 위해 optionalDependencies 명시	Ready 23s	Production	99d97f2	main				May 22	
upload#10: webplatform_w10/client/package.json - Vercel(git) 환경 EBADPLATFORM 충돌 해결을 위해 must 바이너리 제거 및 optionalDependencies로 이동	Error 14s	Production	db677f3	main				May 22	
upload#10: webplatform_w10/client/package.json - Vercel 캐시 무효화 및 리눅스 바이너리 강제 설치를 위한 의존성 명시	Error 11s	Production	2266ff6	main				May 22	
upload#10: Vercel 강제 빌드 트리거를 빈 커밋	Error 5s	Production	11758ef	main				May 22	
upload#10: webplatform_w10/client/package.json - 불필요한 바이너리 다운로드 스크립트 삭제	Error 11s	Production		Redeploy of ALqV52K				May 22	
upload#10: webplatform_w10/client/package.json - 불필요한 바이너리 다운로드 스크립트 삭제	Error 13s	Production		Redeploy of Nb8JqvFCK				May 22	
upload#10: webplatform_w10/package-lock.json - Vercel Rollup 리눅스 바이너리 누락 오류 해결을 위해 윈도우 기반 종속성 잠금 파일 삭제	Error 5s	Production	0251b62	main				May 22	

webplatform

Neon / syncrig-db

syncrig-db

Neon ID: jolly-back-23170908

Connect to Project

Open in Neon

All Databases

Installation

Neon

Status

Created

Plan

Period Total

Available

12h ago

Free

See Installation

Quickstart

env/local

psql

Neon serverless driver

node-postgres

postgres.js

Drizzle

Prisma

Neon Auth

Show secret

Copy Snippet

1

Recommended for most uses

2

DATABASE_URL=*****

3

4

For uses requiring a connection without pgbouncer

5

DATABASE_URL_UNPOOLED=*****

6

7

Parameters for constructing your own connection string

8

PGHOST=*****

SEO

```
<!-- — Primary SEO Meta — -->
<title>SYNCRIG - AI 기반 하드웨어 최적화 그래픽 세팅 추천 플랫폼</title>
<meta name="description" content="SYNCRIG은 PC 하드웨어 사양을 분석하여 가장 적합한 인게임 그래픽 세팅을 추천하는 AI 기반
<meta name="keywords" content="게임 최적화, 그래픽 설정, 하드웨어 매칭, GPU 벤치마크, FPS 최적화, PC 사양, SYNCRIG, 인게임
<meta name="author" content="SYNCRIG Project Team" />
<meta name="robots" content="index, follow" />
<link rel="canonical" href="https://syncrig.vercel.app/" />

<!-- — Open Graph (Facebook, KakaoTalk, Discord 등) — -->
<meta property="og:type" content="website" />
<meta property="og:title" content="SYNCRIG - AI 기반 하드웨어 최적화 플랫폼" />
<meta property="og:description" content="PC 사양을 입력하면 AI 매칭 엔진이 최적의 인게임 그래픽 세팅을 추천합니다." />
<meta property="og:url" content="https://syncrig.vercel.app/" />
<meta property="og:site_name" content="SYNCRIG" />
<meta property="og:locale" content="ko_KR" />

<!-- — Twitter Card — -->
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="SYNCRIG - AI 기반 하드웨어 최적화 플랫폼" />
<meta name="twitter:description" content="PC 사양을 입력하면 AI 매칭 엔진이 최적의 인게임 그래픽 세팅을 추천합니다." />
```

SYNCRIG - AI 기반 하드웨어 최적화 플랫폼

PC 사양을 입력하면 AI 매칭 엔진이 최적의 인게임 그래픽 세팅을...

syncrig.lgw323.com

<https://syncrig.lgw323.com/>

SYNCRIG

SYNCRIG - AI 기반 하드웨어 최적화 플랫폼

PC 사양을 입력하면 AI 매칭 엔진이 최적의 인게임 그래픽 세팅을 추천합니다.

```
<!-- — JSON-LD Structured Data (Schema.org) — -->
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "WebApplication",
  "name": "SYNCRIG",
  "url": "https://syncrig.vercel.app/",
  "description": "AI 기반 PC 하드웨어 최적화 그래픽 세팅 추천 플랫폼",
  "applicationCategory": "GameApplication",
  "operatingSystem": "Web Browser",
  "inLanguage": "ko",
  "offers": {
    "@type": "Offer",
    "price": "0",
    "priceCurrency": "KRW"
  },
  "creator": {
    "@type": "Organization",
    "name": "SYNCRIG Project Team"
  }
}
</script>
```

2.1. SEO (검색엔진최적화)

2.1.1. HTML 메타태그 전면 강화

- [index.html](#):
 - Open Graph 메타태그: og:type, og:title, og:description, og:url, og:site_name, og:locale 총 6종 추가. 카카오톡/디스코드/페이스북 등 소셜 미디어에서 링크 공유 시 풍부한 미리보기 카드를 렌더링합니다.
 - Twitter Card 메타태그: twitter:card, twitter:title, twitter:description 3종 추가. 트위터(X)에서 대형 이미지 카드 형식으로 표시됩니다.
 - Primary SEO Meta: <meta name="keywords"> (게임 최적화, 그래픽 설정, 하드웨어 매칭 등 8개 키워드), <meta name="author">, <meta name="robots" content="index, follow"> 추가.
 - Canonical URL: <link rel="canonical" href="https://syncrig.vercel.app/" /> 로 검색엔진에 정규 URL을 명시하여 중복 콘텐츠 문제를 방지합니다.
 - Theme Color: <meta name="theme-color" content="#0b0f19"> 로 모바일 크롬/사파리 상단 바 색상을 브랜드 컬러와 일치시켰습니다.
 - Favicon: 🐼 이모지 기반 SVG favicon을 인라인으로 설정하여 별도 파일 없이 아이콘을 표시합니다.
 - <noscript> 대체 콘텐츠: JavaScript가 비활성화된 환경(검색엔진 크롤러 포함)에서도 서비스 이름과 설명을 노출합니다.

2.1.2. 구조화 데이터 (JSON-LD / Schema.org)

- [index.html](#):
 - <script type="application/ld+json"> 으로 Schema.org의 WebApplication 타입 구조화 데이터를 삽입했습니다.
 - Google 검색 결과에서 앱 이름, 설명, 카테고리(GameApplication), 가격(무료) 등의 리치 스니펫을 노출할 수 있습니다.

2.1.3. SPA 동적 타이틀 관리

- [useSEO.js](#) (신규 파일):
 - React 커스텀 훅으로, 탭(페이지) 전환 시 document.title 과 <meta name="description">, OG/Twitter 메타태그를 동적으로 업데이트합니다.
 - SPA는 하나의 HTML 파일로 구동되므로, 클라이언트 사이드에서 메타 정보를 갱신하지 않으면 모든 페이지가 동일한 타이틀을 갖게 되는 문제를 해결합니다.
 - 각 탭별 고유 타이틀 매핑: 대시보드(SYNCRIG - 대시보드 | 게임 통계 및 하드웨어 요약), 하드웨어 프로필(SYNCRIG - 하드웨어 프로필 | PC 사양 관리), 최적화 허브(SYNCRIG - 최적화 허브 | AI 그래픽 세팅 추천), 환경 설정(SYNCRIG - 환경 설정 | 테마 · 언어 · 계정).

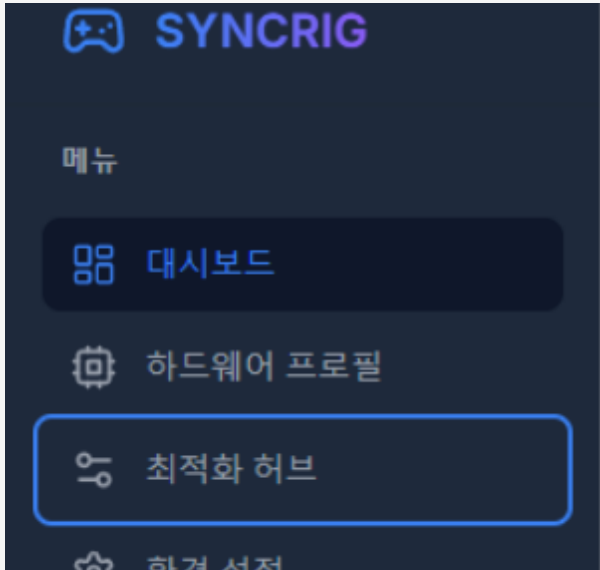
2.1.4. 크롤러 정책 파일

- [robots.txt](#) (신규 파일): 검색엔진 크롤러에게 /api/ 경로를 제외한 전체 페이지의 색인을 허용하고, 사이트맵 위치를 알려줍니다.
- [sitemap.xml](#) (신규 파일): XML 사이트맵 프로토콜 형식으로 SPA 메인 URL을 등록했습니다.

HTML 메타태그 강화

JSON-LD 구조화 데이터

웹 접근성



```
/* 스크린리더 전용 데이터 테이블 (차트 대체 텍스트) */
<table className="sr-only" aria-label="최근 7일간 플레이 시간 데이터">
  <thead>
    <tr>
      <th scope="col">요일</th>
      <th scope="col">플레이 시간</th>
    </tr>
  </thead>
  <tbody>
    {weeklyData.map(d => (
      <tr key={d.day}>
        <td>{d.label}요일 {<{d.day}></td>
        <td>{d.h}시간</td>
      </tr>
    ))}
  </tbody>
</table>
```

```
/* — 웹접근성: 키보드 포커스 링 (focus-visible) — */
*:focus {
  outline: none;
}
*:focus-visible {
  outline: 2px solid #3b82f6;
  outline-offset: 2px;
  border-radius: 4px;
}
```

2.2. 웹접근성 (Web Accessibility, WCAG 2.1)

2.2.1. 스킵 내비게이션 (Skip Navigation)

- [App.jsx](#):
 - 화면 최상단에 “본문으로 건너뛰기” 링크를 추가했습니다. 평소에는 시각적으로 숨겨져 있지만, 키보드 사용자가 Tab 키를 누르면 화면 상단에 표시되어 반복적인 네비게이션을 건너뛸 수 있습니다.
 - → <main id="main-content"> 연결 구조입니다.

2.2.2. 시맨틱 HTML 태그 전환

- [App.jsx](#):
 - 각 탭(대시보드, 하드웨어, 최적화, 설정)의 콘텐츠 영역을 <section aria-labelledby="heading-xxx"> 로 래핑하여 스크린리더가 페이지 구조를 파악할 수 있게 했습니다.
 - 사이드바 <nav> + aria-label="사이드바 메뉴", 모바일 하단 <nav> + <main id="main-content" tabIndex="-1"> 으로 본문 영역을 프로
- [HardwareProfileForm.jsx](#):
 - 폼 단계를 <div> → <fieldset> + <legend> 로 전환하여 논리적
 - 스택 표시를 <div> → <dl><dt><dd> 정의 목록으로 변경하여 사
- [DashboardCharts.jsx](#):
 - 게임 목록을 <div> → 시맨틱 리스트로 전환했습니다.

2.2.4. 폼 접근성 (Label-Input 연결)

- [App.jsx \(AuthScreen\)](#):
 - <label htmlFor="auth-username"> ↔ <input id="auth-username"> 명시적 연결.
 - <label htmlFor="auth-password"> ↔ <input id="auth-password"> 명시적 연결.
 - autoComplete="username" / autoComplete="current-password" 속성으로 브라우저 자동완성을 지원합니다.
- [HardwareProfileForm.jsx](#):
 - 5개 입력 필드 모두 <label htmlFor> ↔ <input id> 연결 완료 (hw-profile-name, hw-cpu, hw-gpu, hw-ram, hw-resolution, hw-refresh).
 - 모든 필수 항목에 aria-required="true" 속성을 부여했습니다.
- [SettingsView.jsx](#):
 - 닉네임 입력: <label htmlFor="settings-nickname"> (sr-only) ↔ <input id="settings-nickname"> 연결.

2.2.3. ARIA 속성 전면 추가

- [App.jsx](#):
 - 인증 화면: 로그인/회원가입 전환을 role="tablist" + role="tab labelledby 로 연결했습니다.
 - 에러 메시지: role="alert" + aria-live="assertive" 로 에러 발
 - 네비 버튼: 활성 탭에 aria-current="page" 속성을 부여하여 현재
 - 알림 벨: aria-label="알림 (새 알림 있음)" + aria-haspopup="t
 - 로딩 상태: role="status" + aria-live="polite" 로 매칭 엔진 진
 - 동기화 오버레이: role="alert" + aria-live="assertive" + ar
 - 장식용 아이콘: 모든 Lucide 아이콘에 aria-hidden="true" 적용하
- [HardwareProfileForm.jsx](#):
 - 프로그레스 바: role="progressbar" + aria-valuenow/min/max
 - 프리셋 영역: role="group" + aria-label="퀵 프리셋 선택" 으로
 - 삭제 버튼: aria-label="\$ {프로필명} 프로필 삭제" 로 각 프로필별
- [RecommendationList.jsx](#):
 - 유사도 바: role="progressbar" + aria-valuenow + aria-label
 - 피드백 버튼: role="group" + 개별 aria-label 로 "도움됨/작동 안
 - 복사 결과: aria-live="polite" 영역으로 감싸 복사 완료 시 스크
- [SettingsView.jsx](#):
 - 테마 토글: role="switch" + aria-checked + aria-labelledby
 - 언어 선택: role="radiogroup" + role="radio" + aria-checked
 - 저장 결과: aria-live="polite" + .sr-only 피드백 영역으로 자

2.2.5. 모달 접근성 (포커스 트랩)

- [RecommendationList.jsx](#):
 - role="dialog" + aria-modal="true" : 모달이 열리면 스크린리더가 이를 대화 상자로 인식합니다.
 - 포커스 트랩: Tab 키를 누르면 모달 내부의 인터랙티브 요소를 사이에서만 순환하며, 모달 밖으로 나가지 않습니다.
 - ESC 닫기: Escape 키를 누르면 모달이 즉시 닫힙니다.
 - 포커스 복원: 모달이 닫히면 모달을 열었던 이전 요소로 포커스가 자동 복원됩니다.
 - 배경 클릭 닫기: 모달 외부(배경 오버레이)를 클릭해도 닫히도록 구현했습니다.

2.2.6. 키보드 내비게이션

- [App.jsx](#):
 - 사이드바 네비게이션에 ↑/↓ 방향키 탐색을 구현했습니다.
 - Home 키로 첫 번째 메뉴, End 키로 마지막 메뉴로 즉시 이동합니다.
 - Enter 또는 Space 키로 메뉴 항목을 선택할 수 있습니다.

2.2.7. 색상 대비 및 시각적 접근성

- [tailwind.config.js](#):
 - WCAG AA 기준(4.5:1 이상) 대비비를 만족하는 커스텀 색상 토큰을 추가했습니다: a11y-muted (#9ca3af, 대비비 약 7.5:1), a11y-subtle (#d1d5db).
 - 기존의 text-gray-500 (#6b7280, 대비비 약 5.0:1로 작은 텍스트에서 불확실)을 text-a11y-muted 로 전면 교체했습니다.
- [index.css](#):
 - 키보드 포커스 링: *:focus-visible 으로 키보드 사용자에게만 파란색 아웃라인을 표시하고, 마우스 사용자에게는 숨깁니다.
 - .sr-only 클래스: 스크린리더 전용 텍스트를 시각적으로 숨기되 스크린리더가 읽을 수 있도록 합니다.
 - 고대비 모드: @media (prefers-contrast: high) 쿼리로 고대비 선호 사용자에게 3px 흰색 아웃라인을 제공합니다.
 - 모션 감소: @media (prefers-reduced-motion: reduce) 쿼리로 모션 감소를 선호하는 사용자에게 모든 애니메이션을 비활성화합니다.

2.2.8. 차트 대체 콘텐츠 (스크린리더 대응)

- [DashboardCharts.jsx](#):
 - 시각적 바 차트에 aria-hidden="true" 를 적용하고, 동일한 데이터를 담은 스크린리더 전용 <table> (.sr-only) 을 제공했습니다.
 - 시각 장애가 있는 사용자도 "월요일: 2.5시간, 화요일: 3.8시간..." 등의 데이터를 테이블 형식으로 접근할 수 있습니다.

(기타)

깃허브 관리

webplatform_w10	upload#10: webplatform_w10/client/src/App.jsx - 대시보드 내...
webplatform_w11	upload#11: webplatform_w11 - 미들웨어 편집 및 단위 테스트 ...
webplatform_w12	upload#12: webplatform_w12 - 산출물 문서(docs) 그룹화 및 배...
webplatform_w2	upload#2: webplatform_w2/README.md - 2주차 기획 및 아이...
webplatform_w3	upload#3: webplatform_w3/README.md - 3주차 서비스 개념 ...
webplatform_w4	upload#4: webplatform_w4/README.md - 4주차 제품 요구사...
webplatform_w5	upload#5: webplatform_w5/README.md - 5주차 UI/UX 프로...
webplatform_w6	upload#6: webplatform_w6/README.md - 6주차 시스템 디자...
webplatform_w7	upload#7: webplatform_w7 - 프론트엔드 UI 컴포넌트 계층형 디...
webplatform_w8	upload#8: webplatform_w8 - 백엔드 서버 인프라 및 DB 모델 계...
webplatform_w9	upload#9: webplatform_w9 - 추천 매칭 알고리즘 MVC 패턴(Ser...
.gitignore	upload#10: .gitignore - 빌드 산출물 및 node_modules 디렉토...
README.md	upload#12: README.md - 1주차부터 12주차까지의 최종 프로젝...

레포지토리 내부

upload#10: client/src/App.jsx - 전역 한글화, 설정 탭 연결 및 모바일 네비게이션 그리드 4칸 분할
upload#10: client/src/features/settings/SettingsView.jsx - 환경 설정 탭 컴포넌트 신규 제작
upload#10: client/src/index.css - 라이트/다크 모드 지원을 위한 CSS Invert 필터 추가
upload#10: webplatform_w10/client/package-lock.json - 하위 잠금 파일 제거 (의존성 충돌 방지)
upload#10: webplatform_w10/package-lock.json - 루트 수준의 통합 의존성 트리(Hoisting) 적용
upload#10: webplatform_w10/package.json - Vercel 서비스 모노레포 빌드를 위한 워크스페이스 환경 설정
upload#10: webplatform_w10/vercel.json - 프론트엔드/백엔드 모노레포 배포용 라우팅 파일 추가
upload#10: webplatform_w10/api/index.js - 백엔드 Vercel Serverless 엔트리포인트 브릿지 생성
upload#10: webplatform_w10/server/server.js - Vercel 서비스 배포를 위한 Express app export 추가
upload#10: webplatform_w10/server/src/controllers/profileController.js - 데이터 무결성을 위한 트랜잭션 동시성 제어 적용
upload#10: webplatform_w10/server/src/config/db.js - 트랜잭션 지원을 위한 DB 커넥션 획득 인터페이스 추가
upload#10: webplatform_w10/server/src/controllers/authController.js - JWT Access Token 만료 시간 단축을 통한 보안 강화
upload#10: webplatform_w10/server/src/middlewares/errorMiddleware.js - winston 기반 구조화된 에러 로깅 도입
upload#10: webplatform_w10/server/server.js - 보안 헤더(helmet) 및 요청 추적 ID(morgan) 로깅 연동
upload#10: webplatform_w10/server/package.json - 로깅 및 보안 미들웨어 패키지 추가

커밋 히스토리

README
SYNCRIG: Hardware Profile Optimization Platform Deployed Vercel Frontend React + Vite Backend Node.js + Express SYNCRIG은 사용자의 PC 하드웨어 사양(CPU, GPU, RAM 등)을 분석하여, 데이터베이스에 축적된 전 세계 게이머들의 최적화 설정 프로파일과 비교하고 가장 적합한 인게임 그래픽 세팅을 추천해 주는 웹 플랫폼입니다. 이 레포지토리는 기획부터 디자인, 알고리즘 구현, 풀스택 연동, 그리고 Vercel 상용 배포까지 이어지는 12주간의 소프트웨어 생명주기(SDLC) 산출물을 모두 담고 있습니다. 주요 기능 (Key Features) 하드웨어 매칭 엔진 (Matching Engine v1.0) - 가중치가 부여된 정규화 L1-노름(Weighted Normalized Manhattan Distance) 기반 알고리즘 탑재 - GPU(50%), CPU(30%), RAM(10%), 해상도(10%) 가중치 기반 유사도(Similarity) 산출 및 랭킹 시스템 통합 대시보드 (Interactive Dashboard) - Recharts를 활용한 스팀/라이엇 게임 플레이타임 및 업적 달성을 게이지 차트 시각화 - 외부 소셜 계정(Steam) 연동 사물레이션 및 데이터 병합(Sync) 기능 구현 풀스택 계층형 아키텍처 (Layered Architecture) - React/Vite 기반의 클라이언트와 Express 기반의 API 서버 분리 (MVC 패턴) - 브라우저 비동기 Fetch 통신 및 JWT 기반 인증 프로세스 구현 기술 스택 (Tech Stack) Frontend - Framework: React 18 (Vite) - Styling: TailwindCSS, PostCSS - Visualization: Recharts, Lucide-React Backend & Algorithm - Environment: Node.js, Express.js - Database: PostgreSQL (pg), In-Memory Mock Fallback

README파일

(레퍼런스)

시스템 기반 환경

시스템 기획 및 정의



구조 시각화 모델링

PlantUML Web Server

핵심 로직 개발



서비스 배포

